RDBMS PROJECT REPORT

CYBERTRACK

Cyber Incident Record Management System

A relational database project developed using MySQL and SQL

Submitted by	Waris Sharma
Roll Number	25202010053
Programme	BCA Cybersecurity
Semester	3rd Semester
Subject	RDBMS
Institution	Lamrin Tech Skills University, Punjab
Database Platform	MySQL 8.0+ / MySQL Workbench

Academic Project Submission

ABSTRACT

CyberTrack is a database project designed to organise cybersecurity incident records in a structured way. I chose this idea because incident information can become difficult to manage when details such as the incident type, severity, assigned investigator, evidence information and status updates are kept separately. The project brings these related details into one relational database.

The database is designed using MySQL and contains seven connected tables: Users, Incident_Types, Severity, Incidents, Investigations, Evidence and Incident_Status_History. Primary keys and foreign keys are used to connect records, while constraints help maintain valid and consistent data. SQL can then be used to add records, retrieve information, combine related tables and prepare summaries.

The current project scope is limited to the database layer. It is an academic model using fictional incident information. It does not detect cyberattacks, scan devices, monitor a live network or automatically respond to incidents. Evidence is represented through metadata rather than uploaded evidence files.

This report explains the idea, objectives, requirements, database design, relationships, normalization approach, SQL operations, testing plan and possible future improvements. Actual execution screenshots and final test results should be added after running the database in MySQL Workbench.

Keywords: RDBMS, MySQL, SQL, cybersecurity incidents, primary key, foreign key, normalization, referential integrity.

ACKNOWLEDGEMENT

I would like to thank my department and faculty for giving me the opportunity to work on this RDBMS project. While preparing CyberTrack, I got the chance to apply database concepts in a topic related to my field of study, Cybersecurity. This project helped me understand how tables, relationships and SQL queries can be used to organise information properly. I am also thankful to everyone whose guidance and learning resources helped me during the planning and design of this project.

TABLE OF CONTENTS

1. Introduction

2. Problem Statement

3. Objectives and Scope

4. Requirements and Technology

5. System Architecture

6. Database Design and ER Diagram

7. Table Description / Data Dictionary

8. Relationships and Referential Integrity

9. Normalization

10. SQL Implementation and Query Examples

11. Testing Plan and Results

12. Limitations and Future Scope

13. Conclusion

14. References

Note: Page numbers may change if screenshots or additional material are inserted.

1. INTRODUCTION

Cybersecurity incidents need to be documented properly so that the important details are not lost. For example, a phishing report may have a reporter, a severity level, an investigator, evidence details and several status updates. If all this information is stored in one large sheet or in separate notes, it can become repetitive and difficult to maintain.

CyberTrack is my attempt to model this process using a relational database. Instead of keeping every detail in one table, the information is divided into tables according to its purpose. The tables are connected through primary and foreign keys. This makes it easier to retrieve the information in a meaningful way using SQL.

The project focuses on database design and implementation concepts taught in RDBMS, such as entities, attributes, relationships, keys, constraints, normalization, joins, aggregate functions, views and transactions.

2. PROBLEM STATEMENT

When incident records are maintained without a proper structure, a few common problems can occur:

- Incident details may be scattered across different files or records, making retrieval slower.
- Investigation progress may not be clear if notes and outcomes are not linked to the correct incident.
- Incident categories and severity labels may be entered inconsistently.
- Evidence information may not be properly associated with the incident it belongs to.
- Previous status changes may be difficult to trace if only the latest status is stored.
- Preparing reports may require manually combining information from multiple places.

CyberTrack addresses these database-level problems by keeping the information in related tables. It provides a structured way to store and query fictional incident records.

3. OBJECTIVES

- To design a relational database for maintaining cybersecurity incident records.
- To store user, incident type and severity information separately.
- To link each incident with its reporter, category and severity.
- To maintain investigation notes and investigator details.
- To store evidence metadata with the related incident.
- To maintain a history of incident status changes.
- To practise SQL queries for filtering, joining, grouping and reporting.

3.1 Project Scope

Included: seven relational tables, primary and foreign keys, data constraints, fictional sample records, SQL retrieval and reporting queries, and database-level transaction handling where implemented.

Not included: live network monitoring, vulnerability scanning, automated attack detection, automatic incident response, a web/mobile interface, user login implementation or actual evidence-file storage.

4. REQUIREMENTS AND TECHNOLOGY

4.1 Software Requirements

- MySQL Server 8.0 or later for creating and managing the relational database.
- MySQL Workbench for writing and executing SQL statements and viewing database objects.
- A Windows, Linux or macOS computer capable of running the MySQL tools.

4.2 Hardware Requirements

A standard personal computer or laptop with sufficient memory and storage to run MySQL Server and Workbench is suitable for this academic project. The project does not require specialised cybersecurity hardware because it works with database records rather than live network traffic.

4.3 Technology Used

MySQL: the relational database management system used for the planned implementation.
SQL: the language used to define tables, insert and update records, and retrieve information.
MySQL Workbench: the graphical tool used to connect to MySQL and execute queries.

5. SYSTEM ARCHITECTURE

CyberTrack follows a simple database-layer architecture. A user or database administrator interacts with the database by writing SQL statements in MySQL Workbench. The SQL operations create and manage records in the seven tables. Queries can then retrieve combined information and prepare reports.

The general flow is:

User / Database Administrator → SQL Queries and Operations → CyberTrack Relational Database → Reports and Information Retrieval

The database stores the records and their relationships. MySQL Workbench is the tool used to interact with it. There is no separate frontend or backend application in the current version.

6. DATABASE DESIGN AND ER DIAGRAM

The ER diagram contains seven entities. Each table has a specific responsibility, and the relationships connect the records without unnecessarily repeating the same information. The diagram below represents the planned relational structure for CyberTrack.

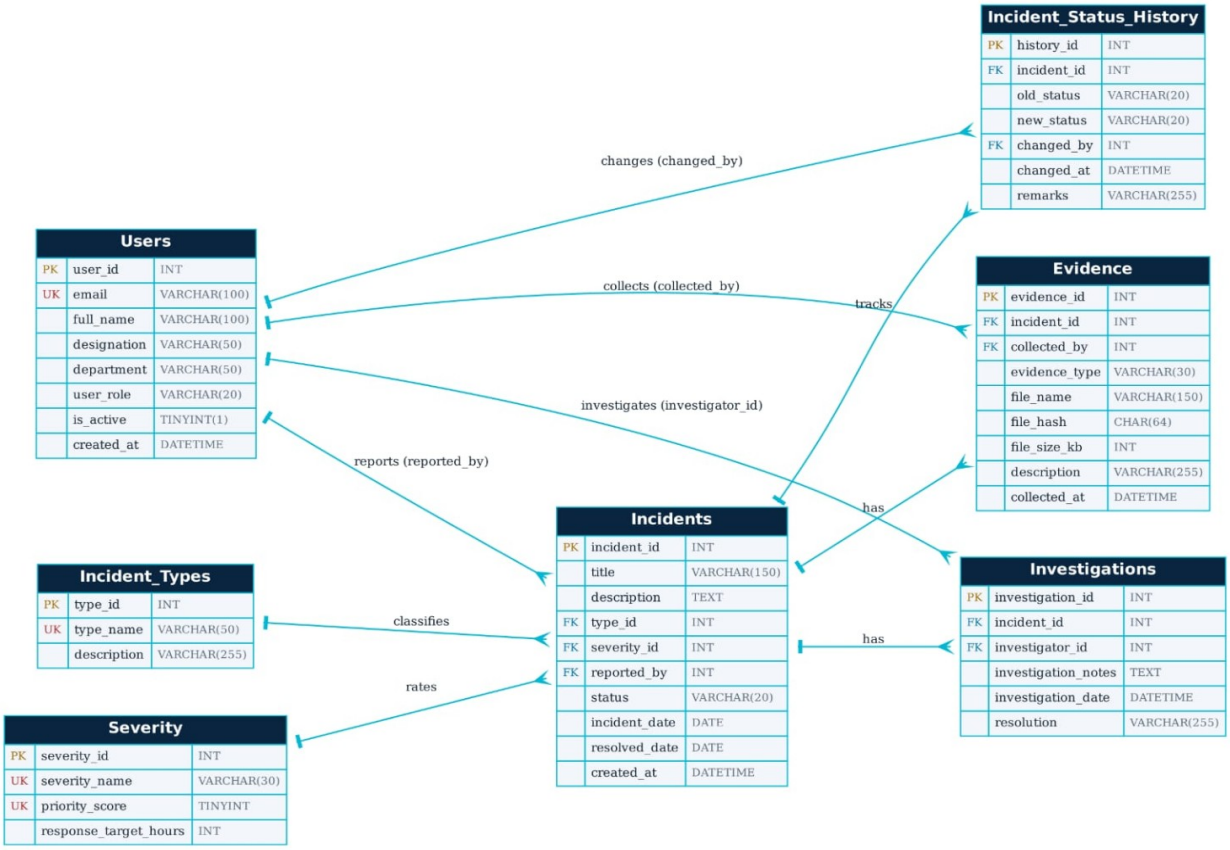


Figure 1: CyberTrack seven-table Entity Relationship Diagram.

7. TABLE DESCRIPTION / DATA DICTIONARY

The database contains seven tables. The following descriptions are based on the submitted ER diagram.

Table	Purpose	Main fields shown in ER diagram
Users	Stores system user details.	user_id (PK), email (unique), full_name, designation, department, user_role, is_active, created_at
Incident_Types	Stores the available incident categories.	type_id (PK), type_name (unique), description
Severity	Stores severity levels and response targets.	severity_id (PK), severity_name (unique), priority_score (unique), response_target_hours
Incidents	Stores the main incident report.	incident_id (PK), title, description, type_id (FK), severity_id (FK), reported_by (FK), status, incident_date, resolved_date, created_at
Investigations	Stores investigation activities and findings.	investigation_id (PK), incident_id (FK), investigator_id (FK), investigation_notes, investigation_date, resolution
Evidence	Stores evidence-related metadata.	evidence_id (PK), incident_id (FK), collected_by (FK), evidence_type, file_name, file_hash, file_size_kb, description, collected_at
Incident_Status_History	Stores changes made to an incident's status.	history_id (PK), incident_id (FK), old_status, new_status, changed_by (FK), changed_at, remarks

7.1 Important Key Fields

PK (Primary Key): uniquely identifies each row in its table. **FK (Foreign Key):** refers to a key in another table and creates a relationship. **Unique field:** prevents duplicate values in the selected column. The exact SQL data types and constraints should match the final executable SQL script.

8. RELATIONSHIPS AND REFERENTIAL INTEGRITY

The ER diagram shows the following one-to-many relationships:

- Users to Incidents: one user can report multiple incidents through reported_by.
- Users to Investigations: one user can be assigned to multiple investigation records through investigator_id.
- Users to Evidence: one user can collect multiple evidence records through collected_by.
- Users to Incident_Status_History: one user can make multiple status-history entries through changed_by.
- Incident_Types to Incidents: one incident type can be used by multiple incidents.
- Severity to Incidents: one severity level can be assigned to multiple incidents.
- Incidents to Investigations: one incident can have multiple investigation records.
- Incidents to Evidence: one incident can have multiple evidence records.
- Incidents to Incident_Status_History: one incident can have multiple status-history entries.

Foreign keys help prevent records from referring to users, incident types, severity levels or incidents that do not exist. This is an important part of maintaining referential integrity.

9. NORMALIZATION

Normalization is the process of organising data into tables to reduce unnecessary duplication and avoid common data anomalies. CyberTrack separates information that has its own identity and purpose. For example, user details are stored in Users rather than being repeated every time that user reports an incident or investigates one.

9.1 First Normal Form (1NF)

The design aims to keep each field atomic, meaning that a column stores one value rather than a list of values. Repeating groups are avoided by creating separate records in tables such as Investigations, Evidence and Incident_Status_History.

9.2 Second Normal Form (2NF)

The tables use single-column primary keys in the submitted ER model. Non-key attributes are intended to describe the whole key. This avoids partial dependency on only part of a composite primary key.

9.3 Third Normal Form (3NF)

The design separates details that belong to different entities. Incident type names and severity descriptions are maintained in their own reference tables, while incident records store their foreign key values. User information is also maintained separately. This reduces transitive dependencies and repeated descriptive data.

10. SQL IMPLEMENTATION AND QUERY EXAMPLES

The database implementation is intended to use Data Definition Language (DDL) statements to create the database and tables, and Data Manipulation Language (DML) statements to insert or modify records. Data Query Language (DQL), mainly SELECT, is used to retrieve information.

10.1 Creating the Database

```
CREATE DATABASE CyberTrack;
USE CyberTrack;
```

10.2 Example Table Definition

The following is a short example showing how a table can be defined. The complete implementation must include all seven tables and the foreign-key relationships shown in the ER diagram.

```
CREATE TABLE Users (
  user_id INT PRIMARY KEY,
  email VARCHAR(100) UNIQUE NOT NULL,
  full_name VARCHAR(100) NOT NULL,
  designation VARCHAR(50),
  department VARCHAR(50),
  user_role VARCHAR(20),
  is_active TINYINT(1) DEFAULT 1,
  created_at DATETIME
);
```

10.3 Example Query: Retrieve Incident Details

```
SELECT i.title, u.full_name, s.severity_name
FROM Incidents AS i
INNER JOIN Users AS u
  ON i.reported_by = u.user_id
INNER JOIN Severity AS s
  ON i.severity_id = s.severity_id;
```

This query combines the incident, user and severity tables. It demonstrates how JOIN operations can retrieve related information without storing all the same details in one table.

10.4 Other SQL Operations Planned

- WHERE and ORDER BY to filter and sort incident records.
- INNER JOIN and LEFT JOIN to combine related tables and identify records with or without matching child entries.
- GROUP BY and aggregate functions such as COUNT to prepare summaries.
- Views to save commonly used SELECT statements for reporting.
- Transactions to group related database changes and use COMMIT or ROLLBACK.

The final query collection should be saved separately in a .sql file and executed in MySQL Workbench. The query examples in this report explain the intended operations; they should not be treated as proof of successful execution.

11. TESTING PLAN AND RESULTS

Testing is necessary to check whether the database structure and its constraints behave as expected. The following checks should be completed in MySQL Workbench after the schema and sample records have been created.

Test	What to check	Evidence to capture
Database selection	Run SELECT DATABASE() after selecting CyberTrack.	Selected database output
Table creation	Run SHOW TABLES and confirm all seven tables appear.	Table list
Table structure	Run DESCRIBE for each table and inspect columns and keys.	DESCRIBE output
Sample records	Run SELECT queries to inspect inserted fictional records.	Sample rows
Relationships	Run JOIN queries to confirm related records connect correctly.	JOIN result grid
Constraints	Try a controlled invalid foreign-key or duplicate value test in a safe test environment.	Error/result message
Reporting	Run GROUP BY and summary queries.	Aggregate result grid
Views / transactions	If implemented, query the view and demonstrate a controlled transaction.	View and transaction output

Testing status: The report has been prepared from the project design and presentation. Actual database execution results and Workbench screenshots were not provided while preparing this PDF. Therefore, this report does not claim that the tests above have passed. The tests should be completed and genuine screenshots inserted before final submission.

11.1 Suggested Screenshot Evidence

- CyberTrack database selected in MySQL Workbench.
- All seven tables visible in the schema panel.
- Table structure showing primary and foreign keys.
- Sample data displayed from the main tables.
- A successful JOIN query with its result grid.
- An aggregate query and, if implemented, view output.
- A controlled transaction or constraint test.

12. LIMITATIONS AND FUTURE SCOPE

12.1 Current Limitations

- The current project is SQL-based and does not include a separate graphical application.
- The database is designed for fictional, manually entered academic records.
- Evidence details are stored as metadata; the actual files are not uploaded or managed.
- There is no implemented login system or role-based access control in the current SQL-only scope.
- It does not perform live monitoring, vulnerability scanning or automatic threat detection.

12.2 Future Improvements

- A web dashboard could be developed to make record entry and reporting easier.
- Role-based access control could be added for different categories of users.
- A secure evidence management feature could be developed to manage evidence files and their metadata.
- Data import and reporting features could reduce manual work.
- The database could later be integrated with approved monitoring or alerting systems, with suitable security controls.

13. CONCLUSION

CyberTrack gave me an opportunity to apply the concepts of RDBMS to a topic connected with Cybersecurity. While designing it, I focused on dividing the information into meaningful tables and connecting those tables through primary and foreign keys. The seven-table design covers users, incident categories, severity levels, incident reports, investigations, evidence metadata and status history.

The main learning from this project is that a good database is not just about storing information. The structure also matters. Proper relationships, constraints and normalization make the data easier to maintain and retrieve. SQL queries such as joins and aggregate functions can then turn the stored records into useful information.

At this stage, CyberTrack should be considered an academic database design and implementation project. Its final working status depends on completing the MySQL execution, inserting sample records and verifying the queries in Workbench. With those steps completed, the project can demonstrate the practical use of relational database concepts in a cybersecurity-related scenario.

14. REFERENCES

- MySQL Documentation — https://dev.mysql.com/doc/
- MySQL Workbench Documentation — https://dev.mysql.com/doc/workbench/en/
- SQLBolt — Interactive SQL Lessons — https://sqlbolt.com/

Project-specific references: the CyberTrack seven-table ER diagram and the CyberTrack RDBMS project presentation prepared for this project.

— END OF REPORT —